



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

CƠ SỞ LẬP TRÌNH

(Programming Basics)

CHAPTER 4: OPERATORS



GVGD: MSc. Van Trung NGUYEN



HỌC KỲ I



KHÓA:

Basic Operations

Java supports the following binary/unary arithmetic operations:

Operator	Mode	Usage	Description	Examples
+	Binary Unary	$x + y$ $+x$	Addition Unary positive	$1 + 2 \Rightarrow 3$ $1.1 + 2.2 \Rightarrow 3.3$
-	Binary Unary	$x - y$ $-x$	Subtraction Unary negate	$1 - 2 \Rightarrow -1$ $1.1 - 2.2 \Rightarrow -1.1$
*	Binary	$x * y$	Multiplication	$2 * 3 \Rightarrow 6$ $3.3 * 1.0 \Rightarrow 3.3$
/	Binary	x / y	Division	$1 / 2 \Rightarrow 0$ $1.0 / 2.0 \Rightarrow 0.5$
%	Binary	$x \% y$	Modulus (Remainder)	$5 \% 2 \Rightarrow 1$ $-5 \% 2 \Rightarrow -1$ $5.5 \% 2.2 \Rightarrow 1.1$

These operators are typically *binary infix* operators, i.e., they take two operands with the operator in between the operands (e.g., $11 + 12$). However, '-' and '+' can also be interpreted as *unary* "negate" and "positive" *prefix* operator, with the operator in front of the operand. For examples,

```
int number = -9;  
number = -number;    // Unary negate
```



Modulus (Remainder) Operator

To evaluate the remainder for negative and floating-point operands, perform repeated subtraction until the *absolute* value of the remainder is less than the *absolute* value of the second operand.

For example,

- $-5 \% 2 \Rightarrow -3 \% 2 \Rightarrow -1$
- $5.5 \% 2.2 \Rightarrow 3.3 \% 2.2 \Rightarrow 1.1$

Exponent?

Java does not have an exponent operator. (The $\wedge$ operator denotes exclusive-or, NOT exponent). You need to use JDK method `Math.exp(x, y)` to evaluate x raises to power y ; or write your own code.



Type Casting

In Java, you will get a compilation "error: incompatible types: possible lossy conversion from double|float|long to int" if you try to assign a double, float, or long value of to an int variable. This is because the fractional part would be truncated and lost. For example,

```
// Assign a "double" value to an "int" variable
double d = 3.5;
int i = d;          //Compilation error: incompatible types: possible lossy conversion from double to i

// Assign a "float" value to an "int" variable
int sum = 55.66f; //Compilation error: incompatible types: possible lossy conversion from float to in

// Assign a "long" value to an "int" variable
long lg = 123;
int count = lg;    //Compilation error: incompatible types: possible lossy conversion from long to int
```



To assign the a double value to an int variable, you need to invoke the so-called *type-casting operator* - in the form of `(int)doubleOperand` - to operate on the double operand and return a *truncated* value in int. In other words, you tell the compiler you consciously perform the truncation and you are fully aware of the "possible lossy conversion". You can then assign the truncated int value to the int variable. For example,

```
double d = 3.5;
int i;
i = (int)d;    // Cast "double" value of 3.5 to "int" 3. Assign the resultant value 3 to i
               // Casting from "double" to "int" truncates.
```

Type casting is an operation which takes one operand. It operates on its operand, and returns an equivalent value in the specified type. The syntax is:

```
(type)variable    // e.g., (int)height
(type)literal     // e.g., (int)55.66
```

```
average = (double)sum / 100;    // Cast sum from int to double before division, double / int -> double
average = sum / (double)100;    // Cast 100 from int to double before division, int / double -> double
average = sum / 100.0;         // int / double -> double / double -> double
average = (double)(sum / 100);  // Won't work. why?
```

Compound Assignment Operators

Besides the usual simple assignment operator (=) described earlier, Java also provides the so-called *compound assignment operators* as listed:

Operation	Mode	Usage	Description	Example
=	Binary	<code>var = expr</code>	Assignment Assign the LHS value to the RHS variable	<code>x = 5;</code>
+=	Binary	<code>var += expr</code> same as: <code>var = var + expr</code>	Compound addition and assignment	<code>x += 5;</code> same as: <code>x = x + 5</code>
-=	Binary	<code>var -= expr</code> same as: <code>var = var - expr</code>	Compound subtraction and assignment	<code>x -= 5;</code> same as: <code>x = x - 5</code>
*=	Binary	<code>var *= expr</code> same as: <code>var = var * expr</code>	Compound multiplication and assignment	<code>x *= 5;</code> same as: <code>x = x * 5</code>
/=	Binary	<code>var /= expr</code> same as: <code>var = var / expr</code>	Compound division and assignment	<code>x /= 5;</code> same as: <code>x = x / 5</code>
%=	Binary	<code>var %= expr</code> same as: <code>var = var % expr</code>	Compound modulus (remainder) and assignment	<code>x %= 5;</code> same as: <code>x = x % 5</code>

Increment/Decrement

Java supports these *unary* arithmetic operators: increment (++) and decrement (--) for all primitive number types (byte, short, char, int, long, float and double, except boolean). The increment/decrement unary operators can be placed before the operand (prefix), or after the operands (postfix). These operators were introduced in C++ to shorthand $x = x + 1$ to $x++$ or $++x$.

Operator	Mode	Usage	Description	Example
++ (Increment)	Unary Prefix Unary Postfix	$++x$ $x++$	Increment the value of the operand by 1. $x++$ or $++x$ is the same as $x += 1$ or $x = x + 1$	<pre>int x = 5; x++; // x is 6 ++x; // x is 7</pre>
-- (Decrement)	Unary Prefix Unary Postfix	$--x$ $x--$	Decrement the value of the operand by 1. $x--$ or $--x$ is the same as $x -= 1$ or $x = x - 1$	<pre>int y = 6; y--; // y is 5 --y; // y is 4</pre>

The increment (++) and decrement (--) operate on its sole operand and store the result back to its operand. For example, $++x$ retrieves x , increment and stores the result back to x .



In Java, there are 4 ways to increment/decrement a variable:

```
int x = 5;
// 4 ways to increment by 1
x = x + 1; // x is 6
x += 1;    // x is 7
x++;       // x is 8
++x;       // x is 9
// 4 ways to decrement by 1
x = x - 1; // x is 8
x -= 1;    // x is 7
x--;       // x is 6
--x;       // x is 5
```

Unlike other unary operator (such as negate (-)) which promotes byte, short and char to int, the increment and decrement do not promote its operand because there is no such need.

The increment/decrement unary operator can be placed before the operand (prefix), or after the operands (postfix), which may affect the outcome.

- If these operators are used by themselves (standalone) in a statement (e.g., `x++`; or `++x`;), the outcomes are the SAME for pre- and post-operators. See above examples.
- If `++` or `--` involves another operation in the SAME statement, e.g., `y = x++`; or `y = ++x`; where there are two operations in the same statement: assignment and increment, then pre- or post-order is important to specify the order of these two operations, as tabulated below:



Operator	Description	Example	Same As
++var (Pre-Increment)	Increment <i>var</i> , and return the incremented <i>var</i> for the other operation in the same statement.	<code>y = ++x;</code>	<code>x = x + 1;</code> <code>y = x;</code>
var++ (Post-Increment)	Return the old value of <i>var</i> for the other operation in the same statement, then increment <i>var</i> .	<code>y = x++;</code>	<code>oldX = x;</code> <code>x = x + 1;</code> <code>y = oldX;</code>
--var (Pre-Decrement)	Decrement <i>var</i> , and return the decremented <i>var</i> for the other operation in the same statement.	<code>y = --x;</code>	<code>x = x - 1;</code> <code>y = x;</code>
var-- (Post-Decrement)	Return the old value of <i>var</i> for the other operation in the same statement, then decrement <i>var</i> .	<code>y = x--;</code>	<code>oldX = x;</code> <code>x = x - 1;</code> <code>y = oldX;</code>



For examples,

```
// Two operations in the statement: increment and assignment
x = 5;
y = ++x;    // Increment x (=6), then assign x to y (=6). (++x returns x+1)
x = 5;
y = x++;    // Assign x to y (=5), then increment x (=6). (x++ returns the oldX)
// After the operations, x gets the SAME value, but the other operation has different outcomes

// Two operations in the statement: increment and println()
x = 5;
System.out.println(++x); // Increment x (=6), then print x (=6). (++x returns x+1)
x = 5;
System.out.println(x++); // Print x (=5), then increment x (=6). (x++ returns the oldX)
```

Notes:

- Prefix operator (e.g., ++i) could be more efficient than postfix operator (e.g., i++)?!
- What is i=i++? Try it out!

Relational and Logical Operators

Very often, you need to compare two values before deciding on the action to be taken, e.g. if mark is more than or equals to 50, print "PASS!".

Java provides six *comparison operators* (or *relational operators*). All these operators are *binary* operators (that takes two operands) and return a boolean value of either true or false.

Operator	Mode	Usage	Description	Example (x=5, y=8)
<code>==</code>	Binary	<code>x == y</code>	Equal to	<code>(x == y) ⇒ false</code>
<code>!=</code>	Binary	<code>x != y</code>	Not Equal to	<code>(x != y) ⇒ true</code>
<code>></code>	Binary	<code>x > y</code>	Greater than	<code>(x > y) ⇒ false</code>
<code>>=</code>	Binary	<code>x >= y</code>	Greater than or equal to	<code>(x >= 5) ⇒ true</code>
<code><</code>	Binary	<code>x < y</code>	Less than	<code>(y < 8) ⇒ false</code>
<code><=</code>	Binary	<code>x <= y</code>	Less than or equal to	<code>(y <= 8) ⇒ true</code>

Take note that the comparison operators are *binary infix* operators, that operate on two operands with the operator in between the operands, e.g., `x <= 100`. It is invalid to write `1 < x < 100` (non-binary operations). Instead, you need to break out the two binary comparison operations `x > 1`, `x < 100`, and join with a logical AND operator, i.e., `(x > 1) && (x < 100)`, where `&&` denotes AND operator.



Java provides four logical operators, which operate on boolean operands only, in descending order of precedence, as follows:

Operator	Mode	Usage	Description	Example
!	Unary	!x	Logical NOT	
&&	Binary	x && y	Logical AND	
	Binary	x y	Logical OR	
^	Binary	x ^ y	Logical Exclusive-OR (XOR)	



The truth tables are as follows:

NOT (!)	true	false
Result	false	true

AND (&&)	true	false
true	true	false
false	false	false

OR ()	true	false
true	true	true
false	true	false

XOR (^)	true	false
true	false	true
false	true	false



Examples:

```
// Return true if x is between 0 and 100 (inclusive)
(x >= 0) && (x <= 100)
// wrong to use 0 <= x <= 100

// Return true if x is outside 0 and 100 (inclusive)
(x < 0) || (x > 100)
// or
!((x >= 0) && (x <= 100))

// Return true if year is a leap year
// A year is a leap year if it is divisible by 4 but not by 100, or it is divisible by 400.
((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0)
```



Exercise

Exercise: Study the following program, and explain its output.

```
/**
 * Test relational and logical operators
 */
public class RelationalLogicalOpTest {
    public static void main(String[] args) {
        int age = 18;
        double weight = 71.23;
        int height = 191;
        boolean married = false;
        boolean attached = false;
        char gender = 'm';

        System.out.println(!married && !attached && (gender == 'm')); //true
        System.out.println(married && (gender == 'f')); //false
        System.out.println((height >= 180) && (weight >= 65) && (weight <= 80)); //true
        System.out.println((height >= 180) || (weight >= 90)); //true
    }
}
```

Write an expression for all unmarried male, age between 21 and 35, with height above 180, and weight between 70 and 80.



- ❑ Let users enter 2 numbers, calculate, subtract, multiply and divide those 2 numbers
- ❑ Let users enter a score of 3 subjects. Let's calculate the GPA and output the results
- ❑ Let users enter value of a, b, c, d, e, g, h variables and output the results:

$$\frac{1 + 2a}{3} + \frac{4(b + c)(5 - d - e)}{f} - 6\left(\frac{7}{g} + h\right)$$



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

